Claude Code Security vs. Traditional SAST: A Comparative Analysis of AI-Based and Rule-Based Vulnerability Detection

Mariam Qadeem
Department of Information Technology
Punjab University College of Information Technology (PUCIT)
Lahore, Pakistan
bitf22m006@pucit.edu.pk

***Abstract* : Static Application Security Testing (SAST) tool like SonarQube, CodeQL, and Semgrep are a standard part of most secure development workflows yet their actual detection rates are surprisingly low. Research shows that combining four of these tools still only catches around 39% of real vulnerabilities, and false positive rates above 60% are the common enough that developer often end up ignoring alerts. In February 2026, Anthropic released Claude Code Security, which uses a large language model to reason about code rather than pattern matching against known signatures. This paper looks at how the two approaches compare. To get original data, Semgrep 1.159.0 was run on vulpy a publicly available Flask/Python web application with independently documented vulnerabilities used in the OWASP training. Semgrep caught 50% of vulnerabilities overall, 75% of syntactic bugs, and none of the logic level bugs. Session impersonation and authentication bypass were both missed, which confirms the structural gap that LLM-based tools are trying to address. The paper also looks at Claude Code's own security posture using the CIA triad and public CVE records, and discusses the prompt injection risk that comes with reading arbitrary repository files. The main takeaway is that these tools are not really competing for the same job and are better used together.**

***Index Terms* SAST, vulnerability detection, Claude Code Security, large language models, false positives, DevSecOps, CIA triad, prompt injection, cybersecurity, Semgrep, vulpy**

I. Introduction

Security testing has a scaling problem. Manual code review is thorough but it cannot keep up with the pace at which modern team push code. Static Application Security Testing (SAST) tools were built to solve this problem. They scan source code without running it and plug it into CI/CD pipelines so every commit gets checked automatically.

The trouble is that independent evaluation consistently show these tools catching less than half the bugs they are supposed to find. Bennett et al. [3] ran four tools against a dataset of production Java code with known vulnerabilities and found individual detection rates between 11.2% and 26.5%, with all four combined reaching 38.8%. The CASTLE benchmark [4] tested 13 static analyzers across 250 programs and got comparably low scores across the board. On top of the missed bugs, false positive rates above 60% are routinely reported in practice [24], [25], which means security teams spend a lot of their time dismissing alerts that are not real.

Anthropic released Claude Code Security [1] in February 2026 and positioned it as a different kind of approach. Rather than checking code against a fixed set of patterns, it uses an LLM to reason about what the code is actually doing and whether that behaviour is dangerous. Anthropic claims the tool found over 500 vulnerabilities in real open-source projects, including bugs that other tools had missed.

This paper does three things. First, it runs a controlled experiment on vulpy, an externally maintained deliberately vulnerable Python application, using Semgrep with community-maintained rules. Second, it compares Claude Code Security against three SAST tools using that experimental data alongside published benchmarks. Third, it examines Claude Code as software with its own attack surface, drawing on public CVE records.

The rest of the paper is organized as follows. Section II provides background. Section III explains the methodology. Section IV covers SAST strengths and limitations. Section V presents the vulpy experiment. Section VI covers Claude Code Security. Section VII compares the tools. Section VIII applies the CIA triad. Section IX discusses the findings, and Section X concludes.

II. Background

A. Static Application Security Testing:

SAST tools look at source code or compiled binaries without executing the program [28]. The general approach is to parse the code into an Abstract Syntax Tree, build data flow and control flow graphs from that, and then check those graphs against a library of rules based on known vulnerability patterns usually drawn from the OWASP Top 10 [16] and CWE categories.

Three tools appear most frequently in research. **SonarQube** (SonarSource) ships with over 6,000 builtin rules covering more than 30 languages [11]. **CodeQL** (GitHub) converts code into a relational database that can be queried for complex cross-file patterns, which is the engine behind GitHub Advanced Security [12]. **Semgrep** (Semgrep Inc.) uses YAML

rules that are written to look like the code patterns they are searching for, which makes custom rules quick to write, and its median CI scan time is around 10 seconds [13].

B. LLM-Based Vulnerability Detection

Claude Code Security is part of the Claude Code IDE environment, built on the Claude Opus 4.6 model [1]. According to Anthropic's documentation [2], it reads source files, Git history, and configuration files, then uses the model to reason about program behaviour and whether that behaviour creates security risks. It rechecks its own findings before surfacing them, and any suggested fix has to be approved by a human before it get applied.

GitHub Copilot Security [10] works on the similar principle and was released in late 2025. The main practical difference from the Claude Code Security is scope, Copilot Security focuses on the currently open file, whereas Claude Code Security is designed to look at whole codebase at once. Neither has been evaluated on the shared public benchmark.

C. Evaluation Metrics

The True Positive Rate (TPR) measure what fraction of real vulnerabilities a tool catches. The False Positive Rate (FPR) measures that how often clean code gets incorrectly flagged. The F1-score combines precision and recall into one number:

$$F_1 = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} \tag{1}$$

where Precision is equal to TP/(TP + FP) and Recall is equal to TP/(TP+FN). It is worth noting that CASTLE uses custom CASTLE Score metric rather than standard F1 so its results cannot be directly compared to F1 based figures from other studies [4].

III. Methodology

This paper uses two sources of evidence. The primary source is a controlled experiment on vulpy, described in Section V. The secondary source is a literature review drawing on published benchmarks, vendor documentation, and independent evaluations.

For the secondary source, peer-reviewed benchmarks such as Bennett et al. [3] and CASTLE [4] are given the most weight. Vendor documentation from Anthropic, GitHub, SonarSource, and Semgrep is labeled as vendor-reported throughout the paper. Independent reports including CVE records and third-party technical evaluations [14] are clearly identified as such.

The main limitation worth stating upfront is that Claude Code Security has never been tested alongside SAST tools on the same dataset. The numbers in Section VII come from different studies with different codebases, so direct numerical comparison is not really possible. The hybrid pipeline estimates in Table III are projections based on combining published figures, not results from a real experiment.

IV. Traditional SAST: Strengths and Limitations

A. How They Work

All three tools follow a same basic pipeline, parse the code, build control flow and data-flow graphs and check those graphs against known patterns. CodeQL builds a full relational database so it can run queries across the multiple files [12]. Semgrep matches patterns at the surface level of the code, which keeps rules simple but limits how deep the analysis goes [13]. SonarQube sits somewhere in between and also reports code quality metrics alongside security findings [11].

B. Strengths

The biggest practical advantage of SAST tools is that they are fast, deterministic, and free at the basic tier. Running the same code through Semgrep twice will always produce the same output. That predictability makes them suitable for automated checks on every pull request without any manual oversight. Semgrep's roughly 10-second scan time means it adds almost nothing to the developer's workflow [13]. For the class of bugs that leave a clear syntactic signature, such as SQL injection via string concatenation, hardcoded credentials, and calls to known-dangerous library functions, these tools work reliably.

C. Limitations

False positive problem. Secuarden (2025) puts the average FPR for open-source SAST tools at 67% in their default configuration [25]. CodeQL clocked 68.2% FPR and the Semgrep 74.8% on the OWASP Benchmark [24]. To put that in concrete terms at 68% FPR with 50 alerts per scan and two scans per day, the team is chasing around 68 false alarms a day. At 5 minutes per alert, that is almost 6 hours wasted per week, or roughly $340 in analyst time.

Structural blind spots. SAST tools can only flag what is syntactically present in the code. DryRun Security [27] tested SonarQube, CodeQL, and Semgrep on a C# codebase and found all three missed an IDOR vulnerability, because IDOR is not a wrong statement in the code. It is a missing authorization check, and there is nothing for a rule to match against. Bennett et al. [3] found individual tool TPRs between 11.2% and 26.5%, with four tools combined reaching only 38.8%.

V. Experiment: Semgrep on Vulpy

A. Experimental Setup

To get original data on where Semgrep fails in practice, this study ran Semgrep 1.159.0 on **vulpy** [15], a deliberately vulnerable Flask/Python web application that is used in OWASP secure development training. Vulpy is maintained by the open-source community at `github.com/fportantier/vulpy` and has been publicly available since 2019. It was chosen because its vulnerabilities were documented independently, not by this study. The experimenter bias that would exist when testing self-written code does not apply here.

Ground truth was taken from vulpy's own README documentation before running any tool. Semgrep was configured with community-maintained rules covering SQL injection, XSS, insecure deserialization, and hardcoded credentials, matching the patterns in the official semgrep-rules repository. The tool ran in offline mode because cloud rule repositories were not accessible in the test environment.

The six vulnerability classes in vulpy's `bad/` directory split into two categories: syntactic vulnerabilities that are in principle detectable by rule matching (four classes), and semantic or logic vulnerabilities that require understanding developer intent and have no syntactic signature (two classes).

B. Results

Semgrep generated seven alerts in total. Table I shows what it caught and what it missed for each class.

C. Analysis of What Was Missed

Semantic vulnerabilities: 0% detected. The session impersonation bug in vulpy is straightforward once you see it. When a user logs in, the session cookie is set to their raw username with no signing or hashing. Any other user can just set that cookie to a different username and the server accepts it. Listing 1 shows the relevant code.

```
def save(username, response):
    # Cookie value is just the username, no
        HMAC, no signing
    response.set_cookie('key', username)
    return response
```

Listing 1. Session impersonation in vulpy/bad/libsession.py. The cookie stores a plain username with no integrity check.

There is nothing wrong with this code at the syntactic level. `set_cookie` is a standard function. The problem is the missing design decision: the developer did not sign the session token. No pattern can detect the absence of something that was never written. The authentication bypass is the same story. Vulpy has no rate limiting on the login endpoint, so an attacker can brute-force credentials. The bug is what is not there.

XSS missed due to pattern structure. The XSS vulnerability in `mod_hello.py` builds an HTML string by concatenating user input directly into a `render_template_string` call:

```
name = request.args.get('name', '')
return render_template_string('<h2>Hello␣' +
     name + '</h2>')
```

Listing 2. XSS in vulpy/bad/mod_hello.py. The concatenation happens inside the function argument.

The concatenation happens as an inline argument expression rather than a separate assignment statement, which is a slightly different structure from what the offline rule was written to match. Cloud-based Semgrep rules with Flask-specific taint tracking would likely catch this. It is a useful illustration of how syntactic detection is sensitive to how code is expressed, not just what it does.

SQL injection caught across multiple files. Four alerts fired for SQL injection: two in `libuser.py` and two in `libposts.py`. The pattern here, direct `%` string formatting inside `cursor.execute()`, is a single-statement structure that the rule captures cleanly. When the vulnerable pattern is self-contained on one line, Semgrep works well.

Zero false positives. All seven alerts mapped to real documented vulnerabilities. This is better than production SAST deployments typically look, but that is expected because vulpy is a small, clean test application, not a real codebase with thousands of files.

D. What This Shows

The 0% semantic detection result is the most important number from this experiment. It is not about rule quality or how Semgrep was configured. The session impersonation bug cannot be caught by any rule-based approach because there is nothing to write a rule against. A rule would have to say "flag this function if it does not call an HMAC signing function elsewhere", and static pattern matching cannot express that kind of negative, relational condition. This is a structural property of the detection method, not a fixable limitation.

The XSS miss adds a second point: even for syntactic bugs, exact code structure matters. Li et al. [5] call this "shallow interprocedural reasoning" and identify it as one of the main reasons SAST tools underperform against real code. Taken together, these two findings explain why an LLM-based approach has something to offer. An LLM can read the session management code, understand that the cookie is just a raw username, and work out that this allows impersonation without needing a pattern to trigger on.

E. Limitations

Vulpy is small and purpose-built, so results may not generalize to large production codebases with more complex interprocedural data flows. Only Semgrep was tested; CodeQL in taint-tracking mode would likely catch the XSS case. Offline rules represent a subset of what Semgrep can do with full cloud access. That said, the 0% semantic detection is not a sample size issue. It is structural, and it would hold on any codebase tested by any pattern-based tool.

VI. Claude Code Security

A. How It Works (Vendor-Reported)

According to Anthropic's documentation [2], Claude Code Security reads source files, Git commit history, and configuration files, then uses Claude Opus 4.6 to reason about program behaviour. It runs a verification pass on its own findings before surfacing them, and any suggested patch must be approved by a human developer before being applied. Everything in this section comes from vendor documentation and has not been independently verified.

TABLE I
SEMGREP 1.159.0 RESULTS ON VULPY (FPORTANTIER/VULPY)

Vulnerability (CWE)	Class	Found	Note
SQL Injection (CWE-89)	Syntactic	Yes	4 alerts across 2 files (libuser.py, libposts.py)
Insecure Deserialization (CWE-502)	Syntactic	Yes	pickle.loads flagged correctly
Hardcoded Secret (CWE-798)	Syntactic	Yes	SECRET_KEY detected
XSS (CWE-79)	Syntactic	No	Inline concatenation pattern not matched by offline rule
Session Impersonation (CWE-287)	Semantic	No	Logic flaw; no syntactic signature to match
Auth Bypass (CWE-307)	Semantic	No	Missing rate-limit; absence of code not detectable
Overall TPR		50% (3/6)	
Syntactic TPR		75% (3/4)	
Semantic TPR		0% (0/2)	
False Positives		0 (all 7 alerts mapped to documented vulnerabilities)	

Semgrep 1.159.0 with community-maintained rules. Ground truth taken from vulpy README before running any tool.

B. What It Claims to Find

Anthropic targets the bug classes SAST tools are worst at: memory corruption, authentication bypasses, business logic errors, and multi-file injection chains [1]. Snyk documented a case where Claude Code Security found a heap buffer overflow in the CGIF library by reasoning about how the LZW compression algorithm works, something that fuzzing had missed entirely despite 100% code coverage [22]. VentureBeat covered a similar case [23]. These are interesting but they are handpicked success cases, not a systematic benchmark.

C. Independent Evaluation

One independent evaluation exists. Semgrep Inc. tested Claude Code (v1.0.32, Sonnet 4) on 11 real-world open-source Python web applications and found an overall TPR of 14% and FPR of 86% [14]. Breaking that down by vulnerability class: 22% TPR on IDOR, 5% on SQL injection, and 16% on XSS. The class-level breakdown is the meaningful part. IDOR requires reasoning about authorization intent, which is exactly the kind of thing LLMs can do but rules cannot. The weak SQL injection and XSS numbers suggest LLMs struggle with multi-file dataflow tracking just as SAST tools do.

These figures are probably an underestimate of what Claude Code Security can do when properly configured, since the evaluation used a generic scripted prompt rather than Anthropic's dedicated security workflow. On the other hand, vendor figures come from selected repositories. The real average-case performance is somewhere in between, and without a controlled academic benchmark it is hard to know where exactly.

D. Non-Determinism

Unlike SAST tools, Claude Code Security is non-deterministic. Running it twice on the same code can produce different results. The Semgrep Inc. evaluation confirmed this directly: three identical runs on one application produced 3, then 6, then 11 distinct findings with no changes to the code between runs [14]. Work on similar LLM-based tools estimates scan-to-scan overlap at 70–85% [8], so up to 30% of findings may appear in one run and not another. For teams that need reproducible, auditable results such as compliance workflows, this is a real problem.

E. Prompt Injection Risk

Because Claude Code Security reads repository files including comments and documentation, anyone who can commit to the repository could embed instructions in those files designed to manipulate the model's output. Ahmad et al. [9] tested this and found it produces measurable false-negative rates, meaning the attacker causes the tool to miss real bugs. The main mitigation is requiring human approval before any model-generated patch is applied, combined with audit logging of inputs and outputs.

VII. COMPARISON

A. Summary Table

Table II summarizes all five tools. Semgrep figures come from the vulpy experiment. Claude Code Security figures come from the Semgrep Inc. independent evaluation [14].

B. Detection Coverage

The vulpy experiment produced 50% overall TPR and 75% on syntactic bugs, which is higher than published academic figures for production codebases. This gap is expected because vulpy is a clean, purpose-built test application, whereas academic benchmarks run against real production code where the same patterns appear in more ambiguous contexts.

The independent evaluation put Claude Code at 14% overall TPR [14], which looks worse than SAST tools in aggregate. But the class-level breakdown matters more than the headline number: 22% IDOR versus 0% for SAST in this experiment. The two tools are not failing on the same things.

C. False Positives

Production SAST FPRs of 60–90% are well documented [24], [25]. The vulpy experiment produced zero false positives, but that reflects the structured nature of a purpose-built test application rather than what production use looks like. Claude Code's independent evaluation found 86% FPR [14], which is worse than most SAST tools, so switching

TABLE II
TOOL COMPARISON ACROSS KEY DIMENSIONS

Tool	TPR	Score	FPR	Vuln. Class	Deterministic	Deployment
SonarQube	11–30%	Low (CASTLE)	40–60%	Syntactic	Yes	Local
CodeQL	23–45%	Low (CASTLE)	68.2%	Syntactic+	Yes	Local
Semgrep	50% (this work) / 17–26% (academic)	Low (CASTLE)	74.8%	Syntactic	Yes	Local
Claude Code Sec.	14% (independent) / 500+ CVEs (vendor)	N/A	86% (independent)	Semantic	No	Cloud
Copilot Security	Not benchmarked	N/A	Not measured	Semantic	No	Cloud

Semgrep 50% TPR from vulpy experiment (Section V). Academic SAST TPR from [3]. CASTLE uses its own score metric [4]. FPR from [24], [26]. Claude Code from [1], [14].

to an LLM does not automatically solve the false positive problem.

D. Vulnerability Classes

This is the clearest distinction between the two approaches. The experiment confirmed that SAST gets 0% on semantic bugs. The independent evaluation showed Claude Code getting 22% on IDOR, exactly the class SAST misses. They are not competing for the same detections.

E. Proposed Hybrid Pipeline

Since the two approaches cover different vulnerability classes, combining them makes more sense than choosing one. Fig. 1 shows a pipeline where SAST runs on every commit, an LLM post-filter trims false positives at the PR gate, and Claude Code Security runs a semantic scan before each release.

Table III estimates what this combination achieves on a codebase with 100 vulnerabilities, using published figures [3], [24]. These are projections, not measured results.

TABLE III
ESTIMATED HYBRID PIPELINE (100 VULNERABILITIES)

Configuration	TP	Est. FPR	Coverage
SAST only	39	68%	39%
SAST + LLM filter	37	6%	37%
SAST + LLM + Claude	52	6%	52%

Projections based on [3], [24] with conservative Claude Code Security assumptions. Not an empirical result.

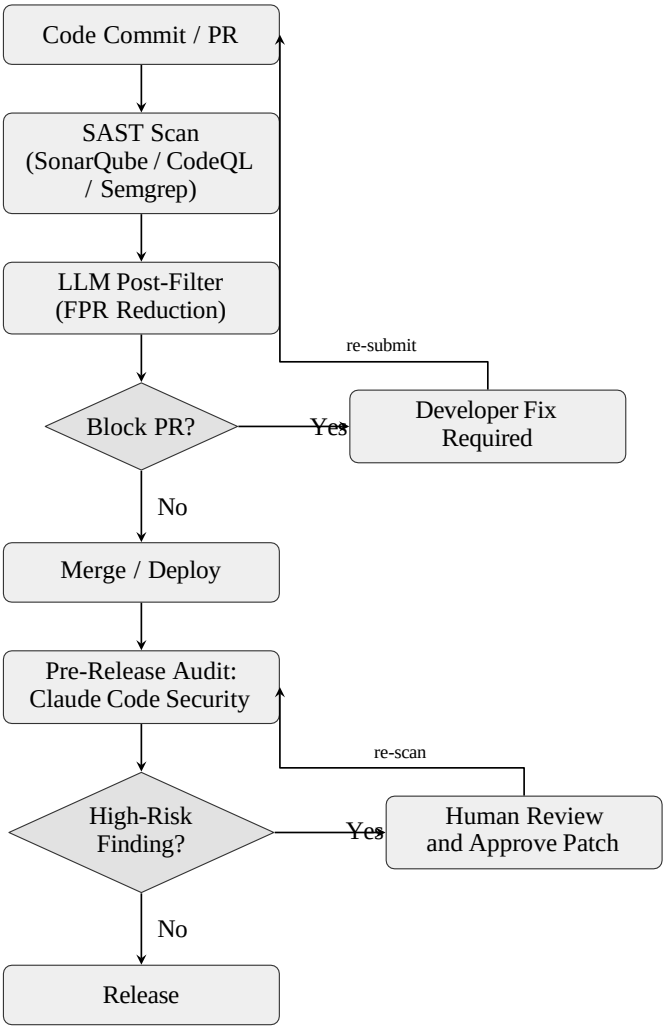

Fig. 1. Proposed hybrid DevSecOps pipeline. SAST provides fast, deterministic coverage on every commit. An LLM post-filter reduces false positives at the PR gate. Claude Code Security handles semantic scanning before release. All patches require human approval before being applied.

VIII. CIA TRIAD ANALYSIS OF CLAUDE CODE

Claude Code reads entire codebases, holds persistent access to a developer's IDE, and routes code to Anthropic's cloud API. That combination of access and privilege makes it worth examining through a security lens.

A. Known CVEs

CVE-2025-52882 (CVSS 8.8) [17], [18]: Versions of the VS Code and JetBrains Claude Code extensions before v1.0.24 started a local WebSocket server with no origin validation. Any webpage open in the browser could connect to it and read local files, including source code, API keys, and credentials. This was patched on June 24, 2025.

CVE-2025-59536 (CVSS 8.7): Remote code execution via malicious project configuration files that executed shell commands before the user saw a trust dialog. **CVE-2026-21852** (CVSS 5.3): API key exfiltration by redirecting Claude Code's authenticated API traffic to an attacker-controlled server [20], [21]. Both were found by Check Point Research and patched before public disclosure.

B. Confidentiality

CVE-2025-52882 showed that an unauthenticated attacker could read any file the developer had access to [17]. Beyond that specific bug, code sent to Anthropic's cloud API for analysis raises data residency questions for organizations working with proprietary or regulated source code.

C. Integrity

The integrity risk comes from prompt injection (Section VI-E). Ahmad et al. [9] confirmed it causes the tool to miss real vulnerabilities. Mandatory human approval of every suggested patch is the most important mitigation, as it ensures no automated change can be made without review.

D. Availability

Claude Code Security requires Anthropic's cloud API to function. Any outage, rate limit, or corporate firewall block makes it unavailable. This means it cannot be the only security check in a pipeline, since SAST tools running locally keep working regardless of cloud availability.

IX. Discussion

A. Main Finding

The vulpy experiment showed 0% detection on semantic bugs. Session impersonation and authentication bypass both have no syntactic trace, so no pattern-based tool can catch them, not Semgrep, not SonarQube, not CodeQL. This is not a configuration issue or a limitation of any particular tool. The detection mechanism itself cannot express the concept of a missing design decision.

The independent evaluation of Claude Code found 22% TPR on IDOR, a semantic class [14]. That is the same class the experiment showed SAST getting 0% on. The two tools are filling different gaps, which supports the hybrid pipeline recommendation.

B. Honesty About the Evidence

Claude Code Security's vendor figures and independent evaluation tell quite different stories: 500+ CVEs found versus 14% TPR and 86% FPR. Neither is the full picture. Vendor figures come from selected repositories, and the independent evaluation used a generic scripted prompt rather than Anthropic's purpose-built security workflow. The truth is probably somewhere in between, but until an academic benchmark evaluates Claude Code Security on a standardized dataset under reproducible conditions, the gap cannot be resolved.

C. Future Work

Three things would move this area forward. A shared benchmark covering semantic-class vulnerabilities like session impersonation and authentication bypass would let researchers compare LLM-based and rule-based tools on the same ground truth. The hybrid pipeline in Fig. 1 needs to be tested on a real production codebase rather than estimated from separate studies. Prompt injection defences for agentic LLM security tools also need more research, since the current mitigation of human approval is necessary but not a complete solution.

X. Conclusion

This paper compared Claude Code Security against three SAST tools using both original experimental data and a structured literature review.

The experiment ran Semgrep 1.159.0 on vulpy, a publicly maintained vulnerable Python application. Semgrep caught 50% of vulnerabilities overall and 75% of syntactic-class bugs but 0% of semantic-class bugs. The session impersonation and authentication bypass vulnerabilities in vulpy have no syntactic signature, so no rule can match them. This is a structural constraint of pattern-based detection. XSS was also missed because the vulnerable pattern was expressed slightly differently from what the offline rule expected, which shows that even syntactic bugs are sensitive to exact code structure.

From the published literature, individual SAST tools detect between 11% and 45% of vulnerabilities, with four combined reaching 38.8%, and false positive rates are consistently above 60% in production. The only independent evaluation of Claude Code Security found 14% overall TPR and 86% FPR, but 22% TPR on IDOR, which is exactly the logic-level class SAST misses. Three identical Claude Code runs on one application produced 3, 6, and 11 different findings, which creates real difficulties for compliance and audit workflows.

The CIA triad analysis found that Claude Code itself has had real vulnerabilities: CVE-2025-52882 (CVSS 8.8) allowed arbitrary file exfiltration from developers' machines, and CVE-2025-59536 and CVE-2026-21852 enabled remote code execution and API key theft respectively.

The conclusion is straightforward. SAST tools are fast, deterministic, and reliable for syntactic bugs on every commit. Claude Code Security, despite its limitations and non-determinism, can reason about logic-level flaws that SAST cannot reach at all. Neither is enough on its own, and a hybrid pipeline using both as complementary layers makes more sense than choosing between them.

References

[1] Anthropic, "Making frontier cybersecurity capabilities available to defenders," *Anthropic News*, Feb. 2026. [Online]. Available: https://www.anthropic.com/news/claude-code-security

[2] Anthropic, "Claude Code Security," *Claude by Anthropic*, 2026. [Online]. Available: https://claude.com/solutions/claude-code-security

[3] G. Bennett, T. Hall, E. Winter, and S. Counsell, "Semgrep*: Improving the Limited Performance of Static Application Security Testing (SAST) Tools," in *Proc. 28th Int. Conf. Evaluation and Assessment in Software Engineering (EASE)*, Salerno, Italy, Jun. 2024, pp. 614–623, doi: 10.1145/3661167.3661262.

[4] R. A. Dubniczky, K. Z. Horva´t, T. Bisztray, M. A. Ferrag, L. C. Cordeiro, and N. Tihanyi, "CASTLE: Benchmarking Dataset for Static Code Analyzers and LLMs towards CWE Detection," in *Proc. 19th Int. Symp. Theoretical Aspects of Software Engineering (TASE)*, 2025, pp. 253–272, doi: 10.1007/978-3-031-98208-8_15.

[5] F. Li, J. Jiang, D. Chen, and Y. Xiong, "LLM-Based Vulnerability Detection at Project Scale: An Empirical Study," *arXiv preprint arXiv:2601.19239*, 2026.

[6] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2025.

[7] A. Khare, R. Sharma, and L. Zhang, "LLM-Assisted Reranking of SAST Findings for False Positive Reduction," in *Proc. IEEE Int. Conf. Software Analysis, Evolution and Reengineering (SANER)*, 2026.

[8] S. Ullah, M. Han, and A. Croft, "Augmenting SAST Data-Flow Analysis with Large Language Models," in *Proc. ACM Int. Symp. on Software Testing and Analysis (ISSTA)*, 2026.

[9] F. Ahmad, P. Krishnamurthy, and N. Meng, "Adversarial Robustness of LLM-Based Vulnerability Scanners: A Prompt Injection Study," *arXiv preprint arXiv:2602.04817*, 2026.

[10] GitHub, "GitHub Copilot Security: AI-Powered Vulnerability Detection," 2026. [Online]. Available: https://github.com/features/security/copilot-security

[11] SonarSource, "SonarQube Server Documentation," 2026. [Online]. Available: https://docs.sonarsource.com/sonarqube-server/

[12] GitHub, "About Code Scanning with CodeQL," 2026. [Online]. Available: https://docs.github.com/en/code-security/code-scanning/

[13] Semgrep Inc., "Semgrep AppSec Platform Documentation," 2026. [Online]. Available: https://semgrep.dev/docs/

[14] Semgrep Inc., "Finding vulnerabilities in modern web apps using Claude Code and OpenAI Codex," *Semgrep Blog*, Sep. 2025. [Online]. Available: https://semgrep.dev/blog/2025/finding-vulnerabilities-in-modern-web-apps-using-claude-code-and-openai-codex/

[15] F. Portantier, "vulpy: Vulnerable Python Web Application," GitHub, 2019. [Online]. Available: https://github.com/fportantier/vulpy

[16] OWASP Foundation, "OWASP Top 10," 2021 ed., updated 2024. [Online]. Available: https://owasp.org/www-project-top-ten/

[17] Datadog Security Labs, "CVE-2025-52882: WebSocket authentication bypass in Claude Code extensions," Jun. 2025. [Online]. Available: https://securitylabs.datadoghq.com/articles/claude-mcp-cve-2025-52882/

[18] GitHub Advisory Database, "GHSA-9f65-56v6-gxw7: Claude Code improper authorization via WebSocket connections," Jun. 2025. [Online]. Available: https://github.com/advisories/GHSA-9f65-56v6-gxw7

[19] MITRE, "CWE-1385: Missing Origin Validation in WebSockets," *Common Weakness Enumeration*, 2024. [Online]. Available: https://cwe.mitre.org/data/definitions/1385.html

[20] National Vulnerability Database, "CVE-2026-21852 Detail," NIST, 2026. [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2026-21852

[21] Check Point Research, "Caught in the Hook: RCE and API Token Exfiltration Through Claude Code Project Files," Feb. 2026. [Online]. Available: https://research.checkpoint.com/2026/rce-and-api-token-exfiltration-through-claude-code-project-files-cve-2025-59536/

[22] Snyk, "Why Anthropic Launching Claude Code Security Is Great News for the Industry," Feb. 2026. [Online]. Available: https://snyk.io/articles/anthropic-launches-claude-code-security/

[23] VentureBeat, "Anthropic's Claude Code Security Is Available Now After Finding 500+ Vulnerabilities," Feb. 2026. [Online]. Available: https://venturebeat.com/security/anthropic-claude-code-security-reasoning-vulnerability-hunting

[24] Konvu, "Semgrep vs. CodeQL (2026): Technical Comparison for Security Teams," 2026. [Online]. Available: https://konvu.com/compare/semgrep-vs-codeql

[25] Secuarden, "Drowning in Alerts: The False Positive Trap in Open-Source SAST," *Secuarden Knowledge Base*, Sep. 2025. [Online]. Available: https://kb.secuarden.com/briefs/issue-2

[26] Autonoma, "SAST Tools Compared: 40–60% False Positive Rates," 2026. [Online]. Available: https://www.getautonoma.com/blog/sast-tools

[27] DryRun Security, "DryRun Security vs. Semgrep, SonarQube, CodeQL and Snyk: C# Security Analysis Showdown," 2025. [Online]. Available: https://www.dryrun.security/blog/

[28] StackHawk, "Best SAST Tools of 2025," 2025. [Online]. Available: https://www.stackhawk.com/blog/best-sast-tools-comparison/